SET A

BATCH 1

SRM Institute of Science and Technology
College of Engineering and Technology
School of Computing
SRM Nagar, Kattankulathur – 603203, Chengalpattu District, Tamilnadu
Academic Year: 2025-26 (ODD)

Test: FJ - II | **Date: 26/09/2025**
Course Code & Title: 21CSC201J Data structures and Algorithm | **Duration:** 1 hour 40 min
Year & Sem: II Year / III Sem | **Max. Marks:** 50

1. **Consider an array arr = [10, 20, 30, 40, 50] with index starting at 0. What is the output of the following loop?**

```
i ← 0
while i < 5 do
        i ← i + arr[i] % 3
        print(arr[i])
end while
```

A. 10 20 30 40 50
B. 20 40 50
C. 10 20 30 50
D. 10 20 30

Step-by-step Execution:

1. **i = 0**
   arr[i] = 10
   10 % 3 = 1 → i = i + 1 = 1
   print(arr[1]) = 20
2. **i = 1**
   arr[i] = 20
   20 % 3 = 2 → i = i + 2 = 3
   print(arr[3]) = 40
3. **i = 3**
   arr[i] = 40
   40 % 3 = 1 → i = i + 1 = 4
   print(arr[4]) = 50
4. **i = 4**
   arr[i] = 50
   50 % 3 = 2 → i = i + 2 = 6
   Now i = 6, which breaks the loop (i < 5 is false)
   Answer: B) 20 40 50

2. **In a Josephus problem with n = 14 people standing in a circle and every second person (k=2) is eliminated, what is the position of the last survivor (1-based indexing)?**
   Last survivor stands at position 13.

3. **What is the net effect on the number of elements in a stack after the following sequence of operations? If any elements remain on the stack; what are they? Initial Stack: Empty**
   Operations:
   Push(10);
   Push(20);
   Pop();
   Push(30);
   Pop();
   Push(40);
   Pop();
   Pop();
   Push(50);
   A. 0 elements; no elements
   B. 1 element; 50 (top)
   C. 2 elements; 40 50 (top)
   D. 1 elements; 10 (top)

4. **What is the postfix expression of the following infix expression:**
   ((A + B) * (C - D)) / ((E ^ F) + G)
   Answer: A B + C D - * E F ^ G + /

Part – B (2 x 5 = 10 Marks)

5. **Write a pseudocode to traverse a singly linked list and determine the number of negative elements present in it.**

```
FUNCTION CountNegativeElements(HEAD)
        COUNT = 0
        PTR ← HEAD

        WHILE PTR ≠ NULL DO
                IF PTR → DATA < 0 THEN
                                COUNT = COUNT + 1
                END IF
                PTR = PTR → NEXT
        END WHILE
        RETURN COUNT
END FUNCTION
```

6. **Consider a Queue Q. Give the algorithm for Enqueue and Dequeue and perform the following operations and show the state of the queue after each operation:**
   Enqueue(A)
   Enqueue(B)
   Dequeue()
   Enqueue(C)
   Dequeue()

Enqueue

Step 1: Allocate memory for the new node and name it as NEWNODE
Step 2: SET NEWNODE → DATA = VAL
Step 3: IF FRONT = NULL
SET FRONT = REAR = NEWNODE
SET FRONT → NEXT = REAR → NEXT = NULL
ELSE
SET REAR → NEXT = NEWNODE
SET REAR = NEWNODE
SET REAR → NEXT = NULL
[END OF IF]
Step 4: END

Dequeue

Step 1: IF FRONT = NULL
Write Underflow
Go to Step 5
[END OF IF]
Step 2: SET PTR = FRONT
Step 3: SET FRONT = FRONT → NEXT
Step 4: FREE PTR
Step 5: END

Operations on Queue

Queue: [ ]
Queue: [A]
Queue: [A, B]
Queue: [B]
Queue: [B, C]
Queue: [C]

Part – C (4 x 8 = 32 Marks)

7. **A modern music streaming app maintains a playlist where users can play, skip, or go back to previously played songs. The playlist is implemented using a suitable data structure, where each node stores a song's title, artist, and duration.**
   - **User starts with 5 songs: *Song A → Song B → Song C → Song D → Song E*.**
   - **The user performs the following operations:**
     a) **Adds *Song F* after Song C.**
     b) **Deletes *Song B* from the playlist.**

   A. Identify the best Data Structure and Draw its Structure (2 Marks)

   B. Write the pseudocode to perform the following operations
   - **Inserting a new song after a given song (3 Marks)**
   - **Delete a Specific Song (3 Marks)**

Data Structure:

The best data structure for this scenario is a Doubly Linked List. This structure allows efficient navigation both forwards (skipping songs) and backwards (going back to previously played songs), as well as efficient insertion and deletion of songs at arbitrary positions within the playlist.

Node Structure

```
STRUCTURE SongNode
   title: STRING
   artist: STRING
   duration: INTEGER
   prev: POINTER TO SongNode
   next: POINTER TO SongNode
END STRUCTURE
```

Inserting a new song after a given song:

```
Function InsertAfter(head, targetTitle, newSong)
   current ← head

   while current ≠ null do
      if current.title = targetTitle then
         newNode ← CreateNode(newSong.title, newSong.artist, newSong.duration)

         newNode.next ← current.next
         newNode.prev ← current

         if current.next ≠ null then
            current.next.prev ← newNode
         end if

         current.next ← newNode
         break
      end if
      current ← current.next
   end while
End Function
```

Delete a Specific Song

```
Function DeleteSong(head, targetTitle)
   current ← head

   while current ≠ null do
      if current.title = targetTitle then
         if current.prev ≠ null then
            current.prev.next ← current.next
         else
            head ← current.next  // deleting head
         end if
```

```
            if current.next ≠ null then
                current.next.prev ← current.prev
            end if

            break
        end if
        current ← current.next
    end while

    return head
End Function
```

8. **Consider the following two polynomials:**
   - **$P_1(x) = 6x^3 + 9x^2 + 7x + 1$**
   - **$P_2(x) = 8x^3 + 5x^2 + 4x + 5$**

   (a) Illustrate the linked list representation for each of the two polynomials. (4 Marks)
   (b) Write pseudocode that adds two such polynomials and returns the resulting polynomial.(4 Marks)

   Illustration of Polynomials
   $P_1(x) = 6x^3 + 9x^2 + 7x + 1$
   [6, 3] → [9, 2] → [7, 1] → [1, 0] → null
   $P_2(x) = 8x^3 + 5x^2 + 4x + 5$
   [8, 3] → [5, 2] → [4, 1] → [5, 0] → null

   Pseudocode to Add Two Polynomials

```
Function AddPolynomials(head1, head2)
    resultHead ← null
    resultTail ← null

    p1 ← head1
    p2 ← head2

    while p1 ≠ null and p2 ≠ null do
        if p1.exponent = p2.exponent then
            sumCoeff ← p1.coefficient + p2.coefficient
            newNode ← CreateNode(sumCoeff, p1.exponent)
            p1 ← p1.next
            p2 ← p2.next

        else if p1.exponent > p2.exponent then
            newNode ← CreateNode(p1.coefficient, p1.exponent)
            p1 ← p1.next

        else
            newNode ← CreateNode(p2.coefficient, p2.exponent)
            p2 ← p2.next
        end if
```

```
        // Append newNode to result list
        if resultHead = null then
            resultHead ← newNode
            resultTail ← newNode
        else
            resultTail.next ← newNode
            resultTail ← newNode
        end if
    end while

    // Append remaining terms
    while p1 ≠ null do
        newNode ← CreateNode(p1.coefficient, p1.exponent)
        resultTail.next ← newNode
        resultTail ← newNode
        p1 ← p1.next
    end while

    while p2 ≠ null do
        newNode ← CreateNode(p2.coefficient, p2.exponent)
        resultTail.next ← newNode
        resultTail ← newNode
        p2 ← p2.next
    end while

    return resultHead
End Function
```

Result

$(6+8)x^3 + (9+5)x^2 + (7+4)x + (1+5)$

$\rightarrow 14x^3 + 14x^2 + 11x + 6$

$\rightarrow [14, 3] \rightarrow [14, 2] \rightarrow [11, 1] \rightarrow [6, 0] \rightarrow$ null

9. **For a Doubly Linked List, design an algorithm that reverses the list in-place so that the first node becomes the last, the second node becomes the second last, and so on. Return the new head of the reversed list. Your pseudocode should clearly show how you handle the prev and next pointers for each node and how you update the head of the list after reversal.**
   Note: An in-place algorithm modifies the data structure directly without using extra space proportional to the input size. For a doubly linked list, this means:
   - ***No new list is created.***
   - ***No auxiliary arrays or stacks.***

```
struct Node *reverse(struct Node *curr) {

    // Base case: If the list is empty or we
      // reach the end of the list
```

```
    if (curr == NULL) return NULL;

    // Swap the next and prev pointers
    struct Node *temp = curr->prev;
    curr->prev = curr->next;
    curr->next = temp;

    // If the previous node (after swap) is NULL
      // his is the new head
    if (curr->prev == NULL) return curr;

    return reverse(curr->prev);
}
```

10. **A theme park ride has a single queue for visitors, but it operates with two rides running simultaneously. Each ride can accommodate one visitor at a time, and visitors are served on a first-come, first-served basis. During a busy hour, the following visitors arrive: V1, V2, V3, V4, V5, V6, V7. The Rules of the ride are:**
    i. **The first two visitors are assigned to the rides immediately.**
    ii. **When a ride becomes free, the next visitor in the queue takes the ride.**
    iii. **Occasionally, a visitor may leave the queue if they get tired of waiting.**
    1. **Draw the queue and ride assignment at each step, showing which visitors are on the rides and who are waiting in line. Assume V3 leaves the queue before taking the ride. (2 Marks)**
    2. **Write pseudocode to:**
        - **Add a new visitor to the queue. (3 Marks)**
        - **Remove a visitor who leaves the queue. (3 Marks)**

Initial Visitors
V1, V2, V3, V4, V5, V6, V7

Step-by-Step Ride & Queue Status

Step	Ride 1	Ride 2	Queue	Remarks
1	V1	V2	V3, V4, V5, V6, V7	V1 & V2 assigned immediately
2	V1	V2	V4, V5, V6, V7	V3 leaves the queue
3	V4	V2	V5, V6, V7	V1 finishes, V4 takes Ride 1
4	V4	V5	V6, V7	V2 finishes, V5 takes Ride 2
5	V6	V5	V7	V4 finishes, V6 takes Ride 1
6	V6	V7	-	V5 finishes, V7 takes Ride 2

Add a Visitor
Step 1: Allocate memory for the new node and name it as NEW_VISITOR
Step 2: SET NEW_VISITOR → ID = VISITOR_ID
Step 3: IF FRONT = NULL
SET FRONT = REAR = NEW_VISITOR
SET FRONT → NEXT = REAR → NEXT = NULL
ELSE

SET REAR → NEXT = NEW_VISITOR
SET REAR = NEW_VISITOR
SET REAR → NEXT = NULL
[END OF IF]
Step 4: END

Remove a Visitor
Step 1: IF FRONT = NULL
Write Underflow
Go to Step 5
[END OF IF]
Step 2: SET VISITOR = FRONT
Step 3: SET FRONT = FRONT → NEXT
Step 4: FREE VISITOR
Step 5: END

11. Evaluate the following postfix expression using a stack.

Postfix Expression:
7 2 3 + - 4 5 * 6 2 / + +

a. Show the contents of the stack after processing each token, clearly indicating the top of the stack.(4 Marks)

b. Write pseudocode that evaluates any valid postfix arithmetic expression.(4 Marks)

Token	Stack (Top of Stack is Right)	Action
7	7	Push
2	7 2	Push
3	7 2 3	Push
+	7 5	Pop 3 & 2 → Push (2+3=5)
-	2	Pop 5 & 7 → Push (7−5=2)
4	2 4	Push
5	2 4 5	Push
*	2 20	Pop 5 & 4 → Push (4×5=20)
6	2 20 6	Push
2	2 20 6 2	Push
/	2 20 3	Pop 2 & 6 → Push (6÷2=3)
+	2 23	Pop 3 & 20 → Push (20+3=23)
+	25	Pop 23 & 2 → Push (2+23=25)

Pseudocode to Evaluate Postfix Expression

```
Function EvaluatePostfix(expression)
  stack ← EmptyStack()

  for each token in expression do
    if token is a number then
      stack.push(token)
    else if token is an operator then
```

```
            operand2 ← stack.pop()
            operand1 ← stack.pop()
            result ← ApplyOperator(operand1, operand2, token)
            stack.push(result)
        end if
    end for

    return stack.pop()
End Function

Function ApplyOperator(op1, op2, operator)
    switch operator
        case '+': return op1 + op2
        case '-': return op1 - op2
        case '*': return op1 * op2
        case '/': return op1 / op2
        case '^': return op1 ^ op2
    end switch
End Function
```

12. **Write a pseudocode to implement Queue data structure using only two Stack (Hint: One Stack can be used as InputStack and one Stack as OutputStack) data structures. The implementation should support enqueue and dequeue operations.**

    Implementation Details:

    - Two Stacks: Maintain two stacks, conventionally named inputStack and outputStack.
    - Enqueue Operation: To enqueue an element, simply push it onto the inputStack. This operation takes O(1) time complexity.
    - Dequeue Operation: To dequeue an element, follow these steps:
        - Check outputStack: If outputStack is not empty, pop the top element from outputStack. This is the element at the front of the queue. This takes O(1) time.
        - Transfer elements: If outputStack is empty, transfer all elements from inputStack to outputStack by repeatedly popping from inputStack and pushing onto outputStack. This reverses the order of elements, effectively bringing the oldest element to the top of outputStack. After the transfer, pop the top element from outputStack. This transfer operation takes O(N) time in the worst case, where N is the number of elements in inputStack.